

Python Installation & Environment Setup



Suman

17 Jan, 2026 At 8:00 PM

Notes

Trimester-1

Recommended

Resource



Associated Lectures (1)



Description



Discussions

SESSION 1.1 LECTURE NOTES

PPT - [Session Intro - Nikhil Sanghi](#)Collab File: [Session_1_1_Practical_Lab](#)In-Class Notes: [Session_1_1_Notes](#)

Python Fundamentals: Variables, Types & Operations

Program: AIM101 — Artificial Intelligence & Machine Learning Foundation **Use:** During and after class reference for students

Mental Map

```
SESSION 1.1: PYTHON FUNDAMENTALS
├── WHY PYTHON?
│   └── #1 language for AI/ML, readable, versatile
├── VARIABLES
│   ├── Named containers for data
│   ├── Assignment with =
│   └── Naming rules (snake_case)
├── DATA TYPES
│   ├── int (whole numbers)
│   ├── float (decimals)
│   ├── str (text)
│   └── bool (True/False)
├── TYPE CONVERSION
│   └── int(), float(), str(), bool()
└──
```

```
└─ Mars Orbiter lesson: types matter!

└─ OPERATORS
  └─ Arithmetic: + - * / // % **
  └─ Comparison: == != < > <= >=
  └─ Logical: and or not

└─ STRINGS
  └─ Indexing: string[0]
  └─ Slicing: string[1:4]
  └─ Methods: .upper(), .lower(), .strip()
  └─ F-strings: f"Hello, {name}"

└─ INPUT/OUTPUT
  └─ print() - display to user
  └─ input() - get from user (ALWAYS returns string!)
```

The Big Picture

Why Are We Learning This?

Every AI/ML program you'll ever write is built on these fundamentals:

- **Variables** store your data (model parameters, dataset values)
- **Types** ensure your calculations are meaningful (the Mars Orbiter cost \$327.6M for ignoring this)
- **Operators** perform computations (every neural network is math operations)
- **Strings** handle text data (NLP, data cleaning)
- **I/O** lets programs interact with users and files

Master these, and everything else becomes easier.

Main Content

1. Variables: Your Data's Home

The Concept

A variable is a **named container** that stores a value. Think of it as a labeled box in a warehouse — you choose the label, Python stores the content.

Creating Variables

```
# Assignment: store value in variable
age = 25
name = "Priya"
height = 5.9
is_student = True
```

Key Rules

Rule	Example	Why It Matters
Must start with letter or underscore	<code>_count</code> , <code>total</code>	Numbers first = error
Case-sensitive	<code>Age</code> $\neq$ <code>age</code> $\neq$ <code>AGE</code>	Python is literal
Use snake_case	<code>student_name</code>	PEP 8 standard
No reserved keywords	Can't use <code>class</code> , <code>if</code> , <code>for</code>	They have special meanings

Reassignment

```
score = 100
print(score) # 100

score = 85    # Old value is GONE
print(score) # 85
```

2. Data Types: The Four Fundamentals

Type Overview

Type	What It Stores	Examples	Check With
<code>int</code>	Whole numbers	<code>42</code> , <code>-17</code> , <code>0</code>	<code>type(42)</code>
<code>float</code>	Decimal numbers	<code>3.14</code> , <code>-0.5</code> , <code>2.0</code>	<code>type(3.14)</code>
<code>str</code>	Text (in quotes)	<code>"Hello"</code> , <code>'Python'</code>	<code>type("Hi")</code>
<code>bool</code>	True or False	<code>True</code> , <code>False</code>	<code>type(True)</code>

The Critical Gotcha

```
# NUMBER addition
5 + 3          # Result: 8

# STRING concatenation
"5" + "3"      # Result: "53"
```

Same symbols, completely different operations! The quotes change everything.

Why Types Matter: The Mars Orbiter

- Lockheed Martin: sent data in **pound-force-seconds**
- NASA: assumed **Newton-seconds**
- Nobody converted → spacecraft entered at wrong altitude → burned up

- **Cost: \$327.6 million**

In Python: `4.45` (float) and `"4.45"` (string) look similar but behave differently. Always check types when debugging!

3. Type Conversion

Conversion Functions

Function	Purpose	Example
<code>int()</code>	Convert to integer	<code>int("42")</code> → 42
<code>float()</code>	Convert to decimal	<code>float("3.14")</code> → 3.14
<code>str()</code>	Convert to text	<code>str(42)</code> → "42"
<code>bool()</code>	Convert to boolean	<code>bool(1)</code> → True

What Works and What Crashes

```
# ✓ WORKS
int("42")      # → 42
float("3.14")  # → 3.14
str(42)        # → "42"

# ✗ CRASHES
int("hello")   # ValueError: can't convert
int("3.14")    # ValueError: has decimal point
float("abc")   # ValueError: not a number
```

Gotcha: `int("3.14")` fails! You must go through float first:

```
int(float("3.14")) # → 3
```

Truncation vs Rounding

```
int(3.9)      # → 3 (truncates, doesn't round!)
int(-3.9)     # → -3 (truncates toward zero)
round(3.9)    # → 4 (if you need rounding)
```

4. Operators

Arithmetic Operators

Operator	Name	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	5 / 2	2.5
//	Floor Division	5 // 2	2
%	Modulo (remainder)	5 % 2	1
**	Power	2 ** 3	8

Division Deep Dive

```
# Regular division always gives float
7 / 2      # → 3.5
4 / 2      # → 2.0 (still float!)

# Floor division rounds DOWN
7 // 2     # → 3
-7 // 2    # → -4 (toward negative infinity)
```

Modulo: The Remainder

```
17 % 5     # → 2 (17 = 5×3 + 2)

# Practical use: check even/odd
number % 2 == 0 # True if even
```

Comparison Operators

Operator	Meaning	Example	Result
==	Equal to	5 == 5	True
!=	Not equal	5 != 3	True
>	Greater than	5 > 3	True
<	Less than	5 < 3	False
>=	Greater or equal	5 >= 5	True
<=	Less or equal	5 <= 3	False

Note: = is assignment, == is comparison!

Logical Operators

Operator	Meaning	Example
and	Both must be True	True and True → True
or	At least one True	True or False → True
not	Flips the value	not True → False

```
age = 25
has_license = True

can_drive = age >= 18 and has_license # True
```

5. Strings

Creating Strings

```
single = 'Hello'
double = "Hello"
triple = '''Multi
line
string'''
```

Use double quotes when string contains apostrophe: "It's working!"

Indexing

```
text = "Python"

# Positive indexing (from start)
text[0]    # → 'P' (first character)
text[1]    # → 'y'
text[5]    # → 'n' (last character)

# Negative indexing (from end)
text[-1]   # → 'n' (last character)
text[-2]   # → 'o'
```

Remember: First character is index 0, not 1!

Slicing

```
text = "Python"

text[0:3]   # → "Pyt" (indices 0, 1, 2 – stop not included!)
text[:3]    # → "Pyt" (from start to 3)
text[3:]    # → "hon" (from 3 to end)
```

```
text[::2] # → "Pto" (every 2nd character)
text[::-1] # → "nohtyP" (reversed)
```

Key Rule: The stop index is NOT included!

Essential String Methods

Method	What It Does	Example
<code>.upper()</code>	ALL CAPS	<code>"hello".upper()</code> → <code>"HELLO"</code>
<code>.lower()</code>	all lowercase	<code>"HELLO".lower()</code> → <code>"hello"</code>
<code>.strip()</code>	Remove whitespace	<code>" hi ".strip()</code> → <code>"hi"</code>
<code>.replace(a, b)</code>	Replace text	<code>"hello".replace("l", "L")</code> → <code>"heLLo"</code>
<code>.split(sep)</code>	Break into list	<code>"a,b,c".split(",")</code> → <code>["a", "b", "c"]</code>
<code>.join(list)</code>	Combine list	<code>"-".join(["a", "b"])</code> → <code>"a-b"</code>

F-Strings (The Modern Way)

```
name = "Raj"
age = 22

# Old way (messy)
"Hello, " + name + "! Age: " + str(age)

# New way (clean)
f"Hello, {name}! Age: {age}"

# Math inside f-strings
f"Next year: {age + 1}"

# Formatting decimals
gpa = 3.847
f"GPA: {gpa:.2f}" # → "GPA: 3.85"
```

6. Input and Output

print(): Talking to Users

```
# Basic
print("Hello!")

# Multiple values
print("Name:", "Raj", "Age:", 25)
# Output: Name: Raj Age: 25

# Custom separator
print("2024", "01", "15", sep="-")
```

```
# Output: 2024-01-15

# No newline at end
print("Loading", end="")
print("...")
# Output: Loading...
```

input(): Listening to Users

```
name = input("Enter your name: ")
print(f"Hello, {name}!")
```

CRITICAL: `input()` ALWAYS returns a string!

```
age = input("Enter age: ") # User types: 25
type(age) # <class 'str'> - it's text, not number!

# To do math, convert first:
age = int(input("Enter age: "))
print(age + 1) # Now this works!
```

The Input Pattern

```
# For text (no conversion needed)
name = input("Enter name: ")

# For integers
age = int(input("Enter age: "))

# For decimals
height = float(input("Enter height: "))
```

Key Frameworks

1. Type Checking Framework

When debugging, always ask:

- What type do I have? (`type(variable)`)
- What type do I need?
- Do I need to convert?

2. The Input-Process-Output Pattern

```
# 1. INPUT: Get data
name = input("Name: ")
age = int(input("Age: "))
```



```
# 2. PROCESS: Do something with it
next_year = age + 1

# 3. OUTPUT: Show results
print(f"Hello {name}, next year you'll be {next_year}")
```

3. String Building Framework

Choose the right approach:

- **Concatenation:** `"Hello, " + name` (simple cases)
- **F-strings:** `f"Hello, {name}"` (preferred for most cases)
- **Join:** `"-".join(items)` (combining lists)

Common Errors Quick Reference

Error	Cause	Fix
<code>NameError: name 'x' is not defined</code>	Variable doesn't exist or misspelled	Check spelling, define before use
<code>TypeError: can only concatenate str to str</code>	Mixing string with number using +	Convert with <code>str()</code> or use f-string
<code>ValueError: invalid literal for int()</code>	Converting non-number text to int	Validate input or use try/except
<code>SyntaxError: invalid syntax</code>	Missing quotes, parentheses, colon	Check syntax carefully
<code>IndexError: string index out of range</code>	Accessing index that doesn't exist	Check <code>len()</code> first

Key Takeaways

Three Things to Remember

Variables are labeled boxes

- You name them, Python stores data
- Reassignment replaces the old value

Types matter

- `5` and `"5"` are completely different
- The Mars Orbiter crashed because of type confusion

- Always check with `type()` when debugging

input() always returns strings

- Convert with `int()` or `float()` when you need numbers
- Pattern: `int(input("prompt"))`

Mini-Project: What We Built

An Interactive Greeting Generator that:

- Gets user's name
- Detects time of day
- Offers greeting styles
- Outputs personalized message

This combined ALL concepts: variables, types, operators, strings, and I/O.

Next Session Preview

Session 1.2: Control Flow & Loops

You'll learn:

- **if/elif/else** — Make decisions in code
- **for loops** — Repeat actions a known number of times
- **while loops** — Repeat until a condition is met
- **Mini-project:** Number Guessing Game

Prepare by:

- Practicing today's concepts
- Completing the assignment
- Thinking about how programs make decisions

Vishlesan i-Hub IIT Patna x Masai School

Breakout Room Recordings:

Aashik: <https://drive.google.com/file/d/1n3qgKb-1ZsCzR4SRjrh3NiqIXCgPOeRZ/view?usp=sharing>

Abhishriya : https://drive.google.com/file/d/122F1cNc4aB4hl-LD3TPxgGZubD90_DNY/view?usp=drive_link

Madhur: https://drive.google.com/file/d/1-ujDSrFd7CF406naYArttG9a25Nyfnfu/view?usp=drive_link